

# PE-EC702C — Neural Network & Fuzzy Logic Control

Semester VII • 3L:0T:0P • 3 Credits

## Module 2 Exam Notes: Competitive Neural Networks

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

---

### Contents

---

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction to Competitive Learning . . . . .</b> | <b>2</b> |
| <b>2</b> | <b>Kohonen Self-Organizing Maps (SOM) . . . . .</b>   | <b>2</b> |
| 2.1      | Kohonen SOM Learning Algorithm . . . . .              | 2        |
| <b>3</b> | <b>Learning Vector Quantization (LVQ) . . . . .</b>   | <b>3</b> |
| 3.1      | LVQ Update Rules . . . . .                            | 3        |
| <b>4</b> | <b>Counter Propagation Networks (CPN) . . . . .</b>   | <b>3</b> |
| 4.1      | CPN Architecture . . . . .                            | 3        |
| 4.2      | CPN Training phases . . . . .                         | 4        |
| <b>5</b> | <b>Worked Examples . . . . .</b>                      | <b>4</b> |

## 1. Introduction to Competitive Learning

Unlike feedforward networks that update weights based on output error signals, competitive learning networks are based on a winner-take-all strategy where nodes compete with each other to become active.

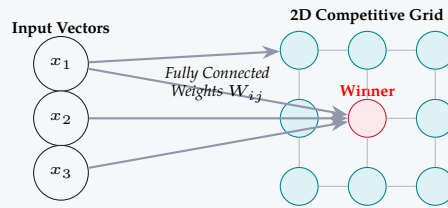
### Competitive Learning

**Competitive Learning** is an unsupervised learning process where output neurons compete for activation. Only one neuron (the winning neuron) remains active at a time, and its weights are updated to become more similar to the input vector.

## 2. Kohonen Self-Organizing Maps (SOM)

Introduced by Teuvo Kohonen in 1982, the Self-Organizing Map (SOM) is an unsupervised learning network that maps high-dimensional input patterns to a low-dimensional (typically 1D or 2D) topological grid of neurons.

### Kohonen Self-Organizing Map Architecture



### 2.1 Kohonen SOM Learning Algorithm

The training process involves four major steps:

1. **Initialization:** Initialize weight vectors  $W_j = (w_{j1}, w_{j2}, \dots, w_{jn})^T$  for each grid node  $j$  with small random values. Set parameters: initial learning rate  $\alpha_0$  and initial neighborhood radius  $\sigma_0$ .
2. **Similarity Matching (Finding the BMU):** For a given input vector  $X = (x_1, x_2, \dots, x_n)^T$ , compute the Euclidean distance to all weight vectors:

$$d_j(X) = \|X - W_j\| = \sqrt{\sum_{i=1}^n (x_i - w_{ji})^2} \quad (1)$$

Identify the winning neuron (Best Matching Unit, BMU) index  $c$ :

$$c = \arg \min_j d_j(X) \quad (2)$$

3. **Weight Updating:** Update weights of the BMU and all neurons within its neighborhood  $h_{cj}(t)$ :

$$W_j(t+1) = W_j(t) + \alpha(t)h_{cj}(t)[X - W_j(t)] \quad (3)$$

4. **Decay Parameters:** Decay the learning rate and the neighborhood size:

$$\alpha(t) = \alpha_0 \exp\left(-\frac{t}{\tau_1}\right), \quad \sigma(t) = \sigma_0 \exp\left(-\frac{t}{\tau_2}\right) \quad (4)$$

### 3. Learning Vector Quantization (LVQ)

Learning Vector Quantization (LVQ), introduced by Kohonen, is a supervised competitive learning algorithm used for pattern classification.

#### LVQ Architecture

An **LVQ network** consists of an input layer and a competitive layer containing codebook vectors (prototype vectors). Each codebook vector is assigned a specific target class.

#### 3.1 LVQ Update Rules

Let  $w_c$  be the codebook vector closest to the input vector  $x$  (in terms of Euclidean distance). The weights are adjusted as follows:

#### LVQ Supervised Training updates

If the winning codebook vector  $w_c$  belongs to the **correct class** as the input  $x$ :

$$w_c(t+1) = w_c(t) + \alpha(t)[x - w_c(t)] \quad (5)$$

If the winning codebook vector  $w_c$  belongs to an **incorrect class** relative to input  $x$ :

$$w_c(t+1) = w_c(t) - \alpha(t)[x - w_c(t)] \quad (6)$$

All other codebook vectors remain unchanged:

$$w_j(t+1) = w_j(t) \quad \text{for } j \neq c \quad (7)$$

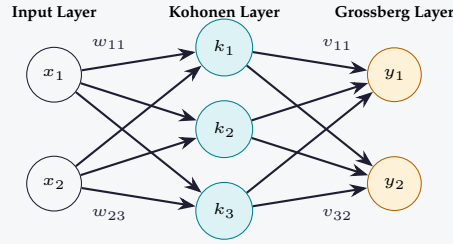
### 4. Counter Propagation Networks (CPN)

Developed by Robert Hecht-Nielsen in 1987, CPN combines a competitive unsupervised layer (Kohonen SOM) with a self-organizing outstar layer (Grossberg outstar). It acts as a fast lookup table to approximate continuous mathematical functions.

#### 4.1 CPN Architecture

CPN contains three layers: Input layer, Kohonen layer (competitive), and Grossberg (Outstar) output layer.

## Counter Propagation Network (CPN) Layout



## 4.2 CPN Training phases

1. **First Phase (Kohonen Training):** The input vector  $X$  is presented. The Kohonen layer acts as a competitive classifier. The winning neuron  $c$  is determined, and its weights  $W_c$  are updated using standard Kohonen rules.
2. **Second Phase (Grossberg Training):** The Kohonen layer outputs a binary vector where only the winning unit has activation 1, and all others are 0 ( $z_j = \delta_{jc}$ ). The Grossberg weights  $V$  are updated to match the target vector  $Y$ :

$$v_{jk}(t+1) = v_{jk}(t) + \beta[y_k - v_{jk}(t)]z_j \quad (8)$$

For the winner  $c$ :  $V_c(t+1) = V_c(t) + \beta[Y - V_c(t)]$ .

## 5. Worked Examples

## Kohonen SOM Winning Neuron and Weight Update

**Problem:** Given an input vector  $X = \begin{bmatrix} 0.8 & 0.6 \end{bmatrix}^T$  and three Kohonen neurons with weight vectors:

$$W_1 = \begin{bmatrix} 0.9 & 0.1 \end{bmatrix}^T, \quad W_2 = \begin{bmatrix} 0.7 & 0.5 \end{bmatrix}^T, \quad W_3 = \begin{bmatrix} 0.2 & 0.8 \end{bmatrix}^T$$

- (a) Identify the winning neuron (BMU) using Euclidean distance.
- (b) Update the weight vector of the winning neuron using learning rate  $\alpha = 0.5$ .

**Solution:**

(a) **Similarity Matching:** Compute the squared Euclidean distance  $D_j = \|X - W_j\|^2$  for each neuron:

- For  $W_1$ :

$$D_1 = (0.8 - 0.9)^2 + (0.6 - 0.1)^2 = (-0.1)^2 + (0.5)^2 = 0.01 + 0.25 = 0.26$$

- For  $W_2$ :

$$D_2 = (0.8 - 0.7)^2 + (0.6 - 0.5)^2 = (0.1)^2 + (0.1)^2 = 0.01 + 0.01 = 0.02$$

- For  $W_3$ :

$$D_3 = (0.8 - 0.2)^2 + (0.6 - 0.8)^2 = (0.6)^2 + (-0.2)^2 = 0.36 + 0.04 = 0.40$$

Since  $D_2 = 0.02$  is the minimum, **Neuron 2** ( $W_2$ ) is the winning neuron (BMU).

**(b) Weight Update:** Update the weights of the winning neuron  $W_2$ :

$$\begin{aligned} W_2(new) &= W_2(old) + \alpha[X - W_2(old)] \\ &= \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} + 0.5 \left( \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix} - \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} \right) \\ &= \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} + 0.5 \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.05 \\ 0.05 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.55 \end{bmatrix} \end{aligned}$$

The updated weight vector is  $W_2 = \begin{bmatrix} 0.75 & 0.55 \end{bmatrix}^T$ .

### LVQ Classifier Updates

**Problem:** Given an input vector  $x = \begin{bmatrix} 0.1 & 0.9 \end{bmatrix}$  belonging to Class 1, and the closest codebook vector  $w_c = \begin{bmatrix} 0.2 & 0.7 \end{bmatrix}$ . Calculate the updated codebook vector for:

- Class of  $w_c$  is Class 1 (correct class)
- Class of  $w_c$  is Class 2 (incorrect class)

Assume learning rate  $\alpha = 0.2$ .

**Solution:**

**(a) Correct Class Update (Push towards input):**

$$\begin{aligned} w_c(new) &= w_c(old) + \alpha[x - w_c(old)] \\ &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} + 0.2 \left( \begin{bmatrix} 0.1 \\ 0.9 \end{bmatrix} - \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} \right) \\ &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} + 0.2 \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.2 - 0.02 \\ 0.7 + 0.04 \end{bmatrix} = \begin{bmatrix} 0.18 \\ 0.74 \end{bmatrix} \end{aligned}$$

**(b) Incorrect Class Update (Pull away from input):**

$$\begin{aligned} w_c(new) &= w_c(old) - \alpha[x - w_c(old)] \\ &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} - 0.2 \left( \begin{bmatrix} 0.1 \\ 0.9 \end{bmatrix} - \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} \right) \\ &= \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} - 0.2 \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.2 + 0.02 \\ 0.7 - 0.04 \end{bmatrix} = \begin{bmatrix} 0.22 \\ 0.66 \end{bmatrix} \end{aligned}$$

## Quick Revision — Module II

Competitive Neural Networks | PE-EC702C

### Key Formulas at a Glance

|                                   |                                                      |
|-----------------------------------|------------------------------------------------------|
| <b>Euclidean Distance in SOM</b>  | $d_j(X) = \sqrt{\sum_{i=1}^n (x_i - w_{ji})^2}$      |
| <b>Winning Neuron (BMU) Index</b> | $c = \arg \min_j d_j(X)$                             |
| <b>Kohonen Weight Update</b>      | $W_j(t+1) = W_j(t) + \alpha(t)h_{cj}(t)[X - W_j(t)]$ |
| <b>LVQ Correct Update</b>         | $w_c(t+1) = w_c(t) + \alpha(t)[x - w_c(t)]$          |
| <b>LVQ Incorrect Update</b>       | $w_c(t+1) = w_c(t) - \alpha(t)[x - w_c(t)]$          |
| <b>Grossberg outstar Update</b>   | $V_c(t+1) = V_c(t) + \beta[Y - V_c(t)]$              |

### One-Line Definitions

- **Kohonen SOM:** An unsupervised clustering network that creates a topology-preserving mapping of high-dimensional input features.
- **Best Matching Unit (BMU):** The neuron in a competitive layer whose weight vector is closest to the current input vector.
- **LVQ:** A supervised competitive network that uses prototype vectors to map inputs to specific classification boundaries.
- **CPN:** A hybrid network that maps input patterns to target vectors rapidly by combining a Kohonen layer with a Grossberg layer.
- **Grossberg Outstar:** A learning unit designed to output a specific analog target pattern when triggered by a Kohonen node.

### Frequently Asked / Viva Questions

1. **What is the primary difference between Perceptron learning and Kohonen learning?**  
Perceptron learning is supervised and minimizes output error feedback. Kohonen learning is unsupervised and uses similarity metrics to cluster features.
2. **How does the neighborhood function  $h_{cj}(t)$  change during SOM training?**  
It starts wide to organize the global structure of the map, and then shrinks exponentially over time to allow fine-tuning of local weights.
3. **Why is LVQ considered a supervised learning algorithm while SOM is unsupervised?**  
SOM groups data based solely on input similarities without class labels. LVQ adjusts its codebook vectors based on class labels to optimize classification boundaries.
4. **Explain the role of Grossberg outstar layer in Counter Propagation Networks.**

**Frequently Asked / Viva Questions (contd.)**

The Kohonen layer maps inputs to a single active winning unit. The Grossberg layer acts as a lookup table, outputting a stored target vector associated with that unit.

**5. What is the significance of the vigilance parameter in competitive networks?**

It controls the sensitivity of node matching. In networks like ART, it determines if a new cluster must be created or if the input fits an existing cluster.

**6. Why does CPN train faster than Backpropagation networks?**

CPN updates only the winning Kohonen neuron and the corresponding Grossberg weights, avoiding expensive error propagation across multiple layers.

**7. What is the purpose of lateral inhibition in competitive networks?**

It allows active neurons to suppress the activation of neighboring competitive neurons, ensuring that only one winning neuron (the BMU) remains active.

**8. How does LVQ-2 differ from standard LVQ-1?**

LVQ-1 updates only the single closest codebook vector. LVQ-2 updates the two closest vectors simultaneously if the input falls near the decision boundary.

**9. What is meant by a topology-preserving mapping in SOM?**

It means that input vectors that are close in the high-dimensional feature space are mapped to neurons that are close to each other on the 2D output grid.

**10. What is the codebook vector in Learning Vector Quantization?**

It is a prototype vector representing a class cluster center in the input space.

**Comparison of Competitive Networks**

| Attribute           | Kohonen SOM                | LVQ                    | CPN                                |
|---------------------|----------------------------|------------------------|------------------------------------|
| Learning Type       | Unsupervised               | Supervised             | Hybrid (Supervised + Unsupervised) |
| Primary Application | Dimensionality reduction   | Pattern classification | Function approximation             |
| Output Layer Grid   | 1D or 2D grid structure    | 1D array of vectors    | 1D array of output nodes           |
| Weight Updates      | Updates winner + neighbors | Updates winner only    | Updates winner only                |